
AUTOMATED SERVER HEALTH MONITORING IN ACTION
ASTRA
ALERT
PASS
AutoOps
Monitor
CHECKS
ALERTS
REPORTS
CPU USAGE
42%
MEMORY USAGE
76%
DISK USAGE
61%
SERVICES PASS 3
3
ALERT 1
3
ALERT 1

Automated Server Health Check & Report System

DevOps Engineering Project — Automation & Scripting

Intermediate | 6–10 Hours | Bash / Shell | Cron Jobs | 100 Marks

■ Overview

Students will build a **shell scripting automation system** that monitors a Linux server's health, logs the results, sends alerts when thresholds are breached, and generates a daily HTML report — simulating real-world DevOps on-call automation tasks. No prior Docker or CI/CD knowledge is required.

■ Learning Objectives

- ✔ Write modular, production-style **Bash scripts**
- ✔ Automate recurring tasks using **cron jobs**
- ✔ Parse and process live system metrics programmatically
- ✔ Generate dynamic **HTML reports** directly from script output
- ✔ Implement threshold-based alerting logic (PASS / ALERT)
- ✔ Organise scripts following a real DevOps project folder structure

■ Project Folder Structure

```
health-monitor/
■■■ monitor.sh           # Main runner script
■■■ checks/
■   ■■■ cpu.sh           # CPU usage check
■   ■■■ memory.sh        # Memory usage check
■   ■■■ disk.sh          # Disk usage check
■   ■■■ services.sh      # Service status check
■■■ reports/
■   ■■■ report.html      # Auto-generated HTML report (output)
■■■ logs/
■   ■■■ health.log       # Persistent timestamped log file
■■■ config.cfg           # Threshold configuration file
```

■■ Tasks Breakdown

Task 1 — Configuration File (config.cfg)

Define threshold values that all scripts will read at runtime:

```
CPU_THRESHOLD=80        # Alert if CPU usage exceeds 80%
MEMORY_THRESHOLD=75     # Alert if memory usage exceeds 75%
DISK_THRESHOLD=90       # Alert if disk usage exceeds 90%
SERVICES="ssh cron nginx"  # Space-separated list of services to monitor
```

Task 2 — Individual Check Scripts (checks/)

Each check script must follow this behaviour:

- Source and read its threshold value from **config.cfg**
- Collect the actual current metric from the live system
- Print a colour-coded **PASS** or **ALERT** status with the current value
- Append a timestamped entry to **logs/health.log**

cpu.sh — CPU Usage

Use **top**, **mpstat**, or **/proc/stat** to capture current CPU utilisation percentage.

memory.sh — Memory Usage

Use **free -m** to calculate used memory as a percentage of total available memory.

disk.sh — Disk Usage

Use **df -h** to check the usage percentage on the root partition **/**.

services.sh — Service Status

Use **systemctl is-active** to verify each service listed in config.cfg is currently running.

Task 3 — Main Runner Script (monitor.sh)

The orchestrator script must:

- Source **config.cfg** at startup to load all threshold values
- Call all 4 check scripts in sequence
- Collect results and print a clean, formatted summary to the terminal
- Trigger HTML report generation at the end of each run

Task 4 — HTML Report Generator

Inside **monitor.sh** or as a separate script, generate **reports/report.html** that includes:

- Report generation timestamp (date and time)
- A **colour-coded results table** — green row for PASS, red row for ALERT
- Current metric value vs configured threshold value for each check
- Last **10 log entries** from health.log displayed at the bottom of the page

Task 5 — Automate with Cron

Schedule **monitor.sh** using crontab with the following jobs:

```
# Run health checks and log every 5 minutes
*/5 * * * * /path/to/health-monitor/monitor.sh >> /path/to/logs/health.log 2>&1

# Generate full HTML report every day at 8:00 AM
0 8 * * * /path/to/health-monitor/monitor.sh --report
```

Submit the output of **crontab -l** as a screenshot.

■ Expected Execution Flow

```
Run monitor.sh
      |
```

```
|----> cpu.sh       ---> PASS / ALERT ---> log entry written
|----> memory.sh    ---> PASS / ALERT ---> log entry written
|----> disk.sh      ---> PASS / ALERT ---> log entry written
|----> services.sh  ---> PASS / ALERT ---> log entry written
          |
          v
Generate reports/report.html  (colour-coded table + last 10 logs)
          |
          v
health.log updated with this run's entries
```

■ Submission Criteria

#	Deliverable	Details
1	All .sh scripts	Well-commented, working scripts for all modules
2	config.cfg	Config file with at least 3 configurable thresholds
3	health.log sample	Log file showing at least 10 timestamped entries
4	report.html	Generated HTML report with colour-coded results table
5	Screenshot 1	Terminal output of monitor.sh running successfully
6	Screenshot 2	report.html opened and displayed in a browser
7	Screenshot 3	crontab -l output showing all scheduled jobs
8	Write-up	One page explaining design decisions and each module's role

■ Grading Rubric

Criteria	Marks
All 4 check scripts work correctly and read from config.cfg	25
Logging with accurate timestamps works across all checks	15
HTML report generated with correct data and colour coding	20
Cron jobs are configured correctly for both schedules	15
Code is modular, readable, and well-commented throughout	15
Write-up clearly explains the system design and decisions	10
TOTAL	**100**

■ Tools & Commands Reference

Core tools students will use:
bash • cron / crontab • awk • grep • sed • df • free • top • mpstat • systemctl • date • tee • curl

Bonus Challenges (Optional)

★ **Email Alert** — use **mail** or **sendmail** to send an email when any metric hits ALERT status

★ **Slack Webhook** — send an ALERT notification to a Slack channel using **curl**

★ **Multi-Server Support** — extend the system to SSH into remote servers and run checks remotely

★ **Historical Graph** — parse health.log and generate an ASCII trend chart using **awk**

Submission Details

Mail - amal.kumar@ipsrsolutions.com

Last Date - 25 April 2026 6.00 PM IST

Anything Support

+91 98476 24538 (whatsapp)